CSC 1300 LAB 7

SPRING 2026

Dinosaur File Parsing

Concepts

- String & Character Functions
- Programmer-Defined Functions
- Input Files
- Output Files

Lab Policies - What you Can't Do/Use in this lab

If you do any of the following, you will receive a 0 on some or all of the assignment and possibly get charged with an academic integrity violation (this is very serious).

1. You may not use code or syntax in your lab assignment **that has not been taught** in CSC 1300 class **without referencing your source**. How do you reference your source? By providing a link/URL in a multi-line comment in the section of code where you use "non-1300" code. Otherwise, this will be considered plagiarism. It is safer to use course materials (slides & example programs) to help you create solutions to the assignments.

2. You may not (ever) use the following in your programming & lab assignments in CSC 1300:
   - while(true) or while(1)
   - goto statements
   - recursion
   - exit()
   - ternary operators
   - Any syntax requiring a '?'
   - lambda functions
   - try-catch

3. You **may not use generative AI (genAI)** **<u>in any way</u>** for this lab assignment. Not sure if something is GenAI? Ask a TA or instructor!

4. You may not share your code with anyone other than your lab instructors/TAs/instructors.

Lab Report

After you have completed this assignment, fill out the lab report and submit your lab report in your zip file along with your source files. Name your screenshot **lab7ReportProof**.
Lab Report Link: **https://tntech.co1.qualtrics.com/jfe/form/SV_6M4wwN1g5Ynv01w**

What to Submit in ilearn when you are done

Create a **zip** file named **username_lab6** containing the following .cpp files and upload it to ilearn in the **Lab 6** assignment.

- `Driver.cpp`
- `Lab7.h`
- `Functions.cpp`
- `dinoDirectory.txt`
- `Lab7ReportProof`

Specifications

You are given a text file that has a data dump of dinosaur data. The format of the data in the file is as follows in this order:

1. Animal Name
2. Length and/or Width of animal
3. Mass/Weight of animal
4. What the animal eats
5. Description and facts about the animal

Each piece of information is separated by a '**#**'.

Your job is to write a program that will read all the information from the file and do the following actions:

1. Count the number of dinosaurs read in.
2. Count how many of the dinosaurs have names that end in 'saurus' and print these dinosaurs to an output text file named "**saurus.txt**"
3. Count the number of dinosaurs who are over 10,000 lbs and print these dinosaurs to an output text file named "**over10Grand.txt**"
4. Print statistics (total dinosaurs, number end in 'saurus', number over 10,000 lbs) in an easy-to-read table (refer to sample output)

The output of this program is short compared to the code required. You can see the sample output at the bottom of this document. Your output should be similar to mine (unless you change the dinosaurData.txt file).

File Organization & Using a Header File

You are going to organize your program in separate files so you can practice with using a header file.

- **Lab7.h** – needs to have comment block at top, include guards, #includes, using namespace std, and the function prototypes for the programmer defined functions
- **Driver.cpp** – needs to have comment block at top, #include "Lab7.h", and the main function
- **Functions.cpp** – needs to have comment block at top, #include "Lab7.h", and all the programmer defined function definitions.

Required Functions

You may not have any global variables – even if they are constant. This is so you must practice sending data to/from functions!

Here are the function prototypes that you will need for your programmer-defined functions below.

You may NOT alter these function prototypes.

```
void printOver10(string dName, string dHeight, string dMass, string dEat, string dDesc);
void printSaurus(string dName, string dHeight, string dMass, string dEat, string dDesc);
bool overTenGrand(string);
```

Main Function

- Read in the filename of the data file from the user. (User will type **dinoDirectory.txt**)
  Note: Do not rename this file. When testing your program, we will type in this exact filename and will expect it will work.

- Open the file. Validate the file can be opened and if not, allow the user to enter in a different filename until the file they enter in can be opened. [Validate with **while** loop!]

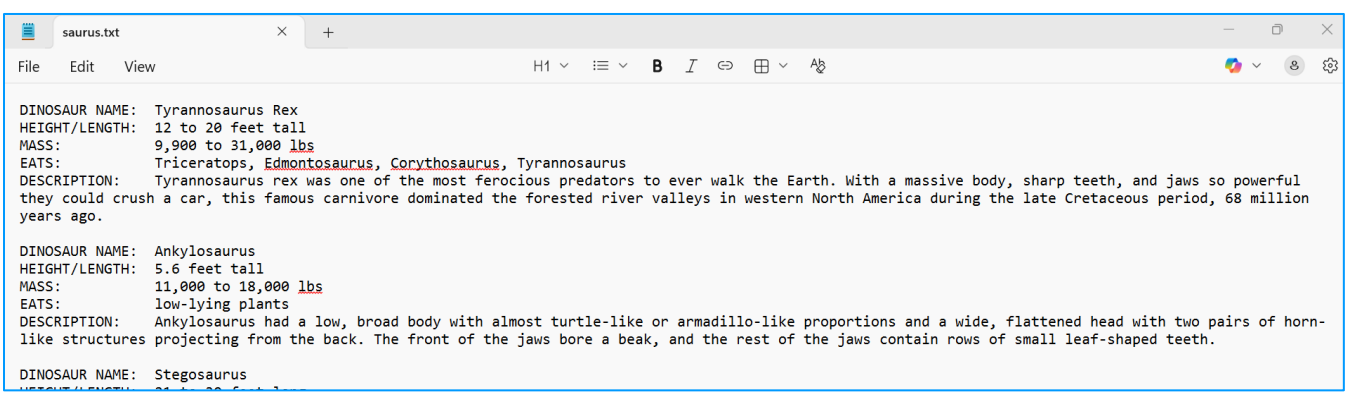
- Start the loop to read from the file. I recommend your loop header be the following line
  `while(getline(dinoDirFile, dinoName, '#'))`
  which will read the animal's name from the file. Inside this loop, you will also:
  - Read in all the other info about the dinosaur (height, weight, what the dinosaur eats, description/facts about dinosaur).
  - Check if the name contain "saurus"? If so, increment the appropriate accumulator variable and then call the **printSaurus** function.
  - Call the **overTenGrand** function, which will return true if the dinosaur is over 10,000 lbs and if so, then increment the accumulator holding this total. Then, call the **printOver10** function.

- Last, print the results to the screen. Refer to the sample output for the format and make sure your output matches the sample output **exactly**. This means you should use stream manipulators to create the columns. The lines above & below the output are made up of 50 dashes.

printSaurus Function

This function prints all the dinosaurs with 'saurus' in the dinosaur name to a text file **saurus.txt**.

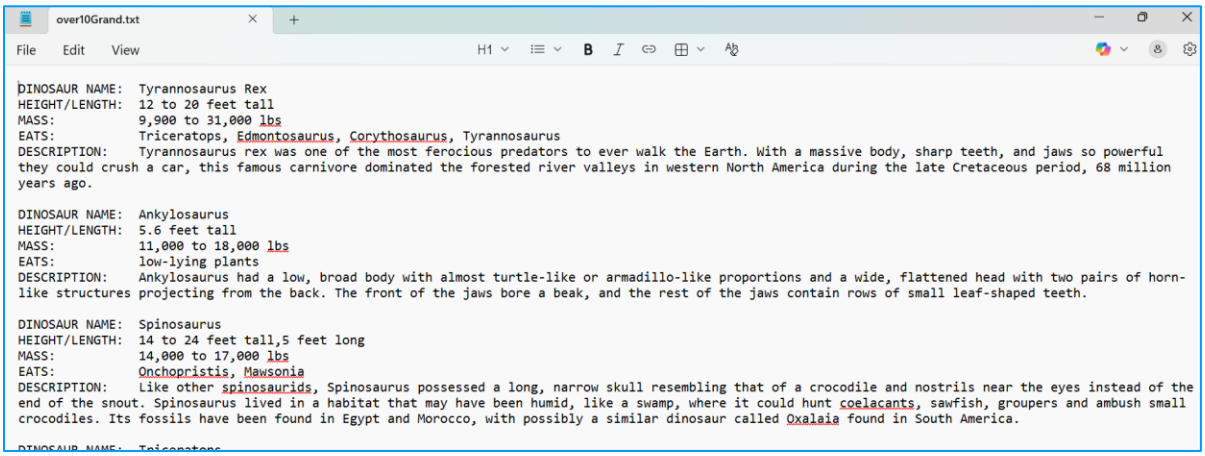
All the dinosaur's data is sent to this function and the function doesn't return anything. Below is a screenshot of what the text file should look like after the program runs. Notice there are labels for each piece of data (like DINOSAUR NAME:) and there is whitespace between each dinosaur printed to aid in readability.



printOver10 Function

This function prints all the dinosaurs who are over 10,000 lbs to a text file **over10Grand.txt**.

All the dinosaur's data is sent to this function and the function doesn't return anything. Below is a screenshot of what the text file should look like after the program runs. Notice there are labels for each piece of data (like DINOSAUR NAME:) and there is whitespace between each dinosaur printed to aid in readability.



```
DINOSAUR NAME:  Tyrannosaurus Rex
HEIGHT/LENGTH:  12 to 20 feet tall
MASS:           9,900 to 31,000 lbs
EATS:           Triceratops, Edmontosaurus, Corythosaurus, Tyrannosaurus
DESCRIPTION:    Tyrannosaurus rex was one of the most ferocious predators to ever walk the Earth. With a massive body, sharp teeth, and jaws so powerful
they could crush a car, this famous carnivore dominated the forested river valleys in western North America during the late Cretaceous period, 68 million
years ago.

DINOSAUR NAME:  Ankylosaurus
HEIGHT/LENGTH:  5.6 feet tall
MASS:           11,000 to 18,000 lbs
EATS:           low-lying plants
DESCRIPTION:    Ankylosaurus had a low, broad body with almost turtle-like or armadillo-like proportions and a wide, flattened head with two pairs of horn-
like structures projecting from the back. The front of the jaws bore a beak, and the rest of the jaws contain rows of small leaf-shaped teeth.

DINOSAUR NAME:  Spinosaurus
HEIGHT/LENGTH:  14 to 24 feet tall,5 feet long
MASS:           14,000 to 17,000 lbs
EATS:           Onchopristis, Mawsonia
DESCRIPTION:    Like other spinosaurids, Spinosaurus possessed a long, narrow skull resembling that of a crocodile and nostrils near the eyes instead of the
end of the snout. Spinosaurus lived in a habitat that may have been humid, like a swamp, where it could hunt coelacants, sawfish, groupers and ambush small
crocodiles. Its fossils have been found in Egypt and Morocco, with possibly a similar dinosaur called Oxalaia found in South America.
```

overTenGrand Function

Parameters: a string, holding the mass of the animal

Returns: a Boolean, true if the animal is over 10,000 lbs and false otherwise

This function is going to challenge you in the problem-solving area (which is why this class is introduction to computer programming AND problem solving)! You may need to use a combination of string class functions and/or character functions to figure out if the animal could be over 10,000 lbs. The string sent to this function could be formatted like one of the following examples:

- `30 to 60 lbs`
- `5,000 to 9,000 lbs`
- `9,000 to 10,000 lbs`
- `11,000 to 15,000 lbs`

The first two examples above should return false and the last two examples should return true. Think about how you can extract out of this string just what you need (without any commas) in order to test the number. You will have to convert the number to an actual integer data type to do a proper comparison.

Some of these pages on cplusplus.com may help with this task:

- http://www.cplusplus.com/reference/string/string/
- http://www.cplusplus.com/reference/string/
- https://www.cplusplus.com/reference/cstdlib/

Code Readability

- Provide a comment block at the top of both source files and the header file that contains the source file name, your name,
- Place a comment above each function telling the function name & purpose.
- Indent your code properly and consistently

Sample Output

```
Dinosaur Directory File Name (dinoDirectory.txt):  dinoDirectory.txt


--------------------------------------------------
               TOTAL DINOS  9
     DINOS OVER 10,000 LBS  6
DINO NAMES END IN 'SAURUS'  7
--------------------------------------------------
```

(and also the **saurus.txt** file and **over10Grand.txt**)